```
Start coding or generate with AI.
```

```
pip install python-docx
```

```
Collecting python-docx
  Downloading python_docx-1.2.0-py3-none-any.whl.metadata (2.0 kB)
Requirement already satisfied: lxml>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from python-docx) (6.0.2)
Requirement already satisfied: typing_extensions>=4.9.0 in /usr/local/lib/python3.12/dist-packages (from python-docx) (4.15.0)
Downloading python_docx-1.2.0-py3-none-any.whl (252 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 253.0/253.0 kB 5.6 MB/s eta 0:00:00
Installing collected packages: python-docx
Successfully installed python-docx-1.2.0
```

Now, let's read the content of the `.docx` file. This code will attempt to extract all text from the document.

```
import docx

doc_path = '/content/EXPERIMENT NO. 5.docx'
doc = docx.Document(doc_path)

full_text = []
for para in doc.paragraphs:
    full_text.append(para.text)

# Join all paragraphs into a single string for display
document_content = '\n'.join(full_text)

print(document_content[:1000]) # Print first 1000 characters to avoid overwhelming output
```

```
EXPERIMENT NO. 5
AIM: Program to implement Unsupervised learning Kohonen’s Self Organizing Feature map (For a problem given and also for MNIST da
THEORY:
A self-organizing map is also known as SOM and it was proposed by Kohonen.
It is an unsupervised neural network that is trained using unsupervised learning techniques to produce a low dimensional, discre
The map retains the calculated relative distance between the points. Points closer to each other within the input space are mapp
Self-Organizing Maps can thus serve as a cluster analysing  tool for high dimensional data.
Self-Organizing Maps also have the capability to generalize.
During generalization, the network can recognize or characterize inputs that it has never seen as data before.
New input is taken up with the map unit and is theref
```

The above output shows the first 1000 characters of the document. If there's specific code within the document, please copy and paste it into a new code cell, or let me know what part you'd like to work with.

```
import numpy as np

# 1. Initialization (Step 0)
# Input vector x = (0.4, 0.07), Learning rate alpha = 0.3
x = np.array([0.4, 0.07])
alpha = 0.1

# Weights from the image (Batch 1)
# Format: [w1, w2] for each output unit Y1 to Y5
weights = np.array([
    [0.7, 0.5], # Y1
    [0.3, 0.2], # Y2
    [0.1, 0.4], # Y3
    [0.6, 0.9], # Y4
    [1.0, 0.5]  # Y5
])

def train_som_step(input_vec, weights, lr):
    # Step 3: Compute Square of Euclidean Distance D(j)
    distances = np.sum((weights - input_vec)**2, axis=1)

    # Step 4: Find winning unit index J (minimum distance)
    winner_idx = np.argmin(distances)
    print(f"Distances D(j): {distances}")
    print(f"Winning Unit: Y{winner_idx + 1}")

    # Step 5: Update weights for the winning unit
    # Formula: w_new = w_old + alpha * (x - w_old)
    old_weight = weights[winner_idx]
```

```
    new_weight = old_weight + lr * (input_vec - old_weight)

    return winner_idx, new_weight

# Execute one iteration
idx, updated_w = train_som_step(x, weights, alpha)
print(f"Updated weights for Y{idx+1}: {updated_w}")
```

```
Distances D(j): [0.2749 0.0269 0.1989 0.7289 0.5449]
Winning Unit: Y2
Updated weights for Y2: [0.33  0.161]
```

```
import numpy as np

# Input vector x based on the user's first 4 elements to match weight dimensions.
# If you intended a different interpretation for 'x', please clarify.
x = np.array([[1, 0, 1, 1],
              [0, 0, 0, 1],
              [1, 0, 0, 1],
              [1, 1, 1, 0]])

# Learning rate alpha = 0.1
alpha = 0.1


weights = np.array([
    [0.9, 0.3, 0.1, 0.2],
    [0.1, 0.6, 0.2, 0.1]
])

def train_som_step(input_vec, weights, lr):


    distances = np.sum((weights - input_vec)**2, axis=1)

    # Step 4: Find winning unit index J (minimum distance)
    winner_idx = np.argmin(distances)
    print(f"Distances D(j): {distances}")
    print(f"Winning Unit: Y{winner_idx + 1}")


    # Formula: w_new = w_old + alpha * (x - w_old)
    old_weight = weights[winner_idx]
    new_weight = old_weight + lr * (input_vec - old_weight)

    return winner_idx, new_weight

# This loop will process each input vector in 'x'
updated_weights_matrix = np.copy(weights)

print("Initial Weights:\n", updated_weights_matrix)

for i, input_vec in enumerate(x):
    print(f"\nProcessing Input Vector {i+1}: {input_vec}")
    idx, updated_w = train_som_step(input_vec, updated_weights_matrix, alpha)
    print(f"Updated weights for Y{idx+1}: {updated_w}")

    # Update the weights matrix for the next iteration
    updated_weights_matrix[idx] = updated_w
    print("Weights matrix after update:\n", updated_weights_matrix)
```

```
Initial Weights:
 [[0.9 0.3 0.1 0.2]
 [0.1 0.6 0.2 0.1]]

Processing Input Vector 1: [1 0 1 1]
Distances D(j): [1.55 2.62]
Winning Unit: Y1
Updated weights for Y1: [0.91 0.27 0.19 0.28]
Weights matrix after update:
 [[0.91 0.27 0.19 0.28]
 [0.1  0.6  0.2  0.1 ]]

Processing Input Vector 2: [0 0 0 1]
Distances D(j): [1.4555 1.22  ]
Winning Unit: Y2
Updated weights for Y2: [0.09 0.54 0.18 0.19]
```

```
Weights matrix after update:
 [[0.91 0.27 0.19 0.28]
 [0.09 0.54 0.18 0.19]]

Processing Input Vector 3: [1 0 0 1]
Distances D(j): [0.6355 1.8082]
Winning Unit: Y1
Updated weights for Y1: [0.919 0.243 0.171 0.352]
Weights matrix after update:
 [[0.919 0.243 0.171 0.352]
 [0.09  0.54  0.18  0.19 ]]

Processing Input Vector 4: [1 1 1 0]
Distances D(j): [1.390755 1.7482  ]
Winning Unit: Y1
Updated weights for Y1: [0.9271 0.3187 0.2539 0.3168]
Weights matrix after update:
 [[0.9271 0.3187 0.2539 0.3168]
 [0.09   0.54   0.18   0.19  ]]
```

```
import numpy as np

# Input vectors (from previous context)
x = np.array([[1, 0, 1, 1],
              [0, 0, 0, 1],
              [1, 0, 0, 1],
              [1, 1, 1, 0]])

# Learning rate (from previous context)
alpha = 0.1

# User specified weight decay factor
# Interpreted as a multiplicative factor applied to weights: W_new = W_old * weight_decay_factor
# A factor of 0.5 means weights are halved after each update step.
weight_decay_factor = 0.5

# Initial weights (from previous context)
weights_initial = np.array([
    [0.9, 0.3, 0.1, 0.2],
    [0.1, 0.6, 0.2, 0.1]
])

def train_som_iteration_with_decay(input_vec, current_weights_matrix, lr, decay_factor):
    """
    Performs one iteration of SOM training for a single input vector,
    including updating the winning unit's weights and then applying
    a global weight decay to the entire weight matrix.
    """
    # Step 3: Compute Square of Euclidean Distance D(j)
    distances = np.sum((current_weights_matrix - input_vec)**2, axis=1)

    # Step 4: Find winning unit index J (minimum distance)
    winner_idx = np.argmin(distances)
    print(f"  Distances D(j): {distances}")
    print(f"  Winning Unit: Y{winner_idx + 1}")

    # Step 5: Update weights for the winning unit (before decay)
    # Formula: w_new = w_old + alpha * (x - w_old)
    old_weight_of_winner = current_weights_matrix[winner_idx]
    new_weight_for_winner_before_decay = old_weight_of_winner + lr * (input_vec - old_weight_of_winner)

    # Create a copy of the current weights to apply updates and decay
    updated_weights_matrix_step = np.copy(current_weights_matrix)
    updated_weights_matrix_step[winner_idx] = new_weight_for_winner_before_decay

    print(f"  Updated weights for Y{winner_idx+1} (before decay applied to matrix): {new_weight_for_winner_before_decay}")

    # Apply weight decay to the entire weight matrix after updating the winning unit
    updated_weights_matrix_step *= decay_factor
    print(f"  All weights in matrix decayed by factor {decay_factor}.")

    return winner_idx, updated_weights_matrix_step, new_weight_for_winner_before_decay


# Start SOM training with weight decay
current_weights = np.copy(weights_initial)

print("Initial Weights:\n", current_weights)
```

```
# Iterate through each input vector
for i, input_vec in enumerate(x):
    print(f"\nProcessing Input Vector {i+1}: {input_vec}")
    # Perform one SOM step with decay
    winner_idx, current_weights, _ = train_som_iteration_with_decay(
        input_vec, current_weights, alpha, weight_decay_factor
    )
    print(f"Weights matrix after processing Input Vector {i+1} and applying decay:\n", current_weights)

print("\nFinal Weights after all input vectors and decays:\n", current_weights)
```

```
Initial Weights:
 [[0.9 0.3 0.1 0.2]
 [0.1 0.6 0.2 0.1]]

Processing Input Vector 1: [1 0 1 1]
  Distances D(j): [1.55 2.62]
  Winning Unit: Y1
  Updated weights for Y1 (before decay applied to matrix): [0.91 0.27 0.19 0.28]
  All weights in matrix decayed by factor 0.5.
Weights matrix after processing Input Vector 1 and applying decay:
 [[0.455 0.135 0.095 0.14 ]
 [0.05  0.3   0.1   0.05 ]]

Processing Input Vector 2: [0 0 0 1]
  Distances D(j): [0.973875 1.005   ]
  Winning Unit: Y1
  Updated weights for Y1 (before decay applied to matrix): [0.4095 0.1215 0.0855 0.226 ]
  All weights in matrix decayed by factor 0.5.
Weights matrix after processing Input Vector 2 and applying decay:
 [[0.20475 0.06075 0.04275 0.113  ]
 [0.025   0.15    0.05    0.025  ]]

Processing Input Vector 3: [1 0 0 1]
  Distances D(j): [1.42470969 1.92625   ]
  Winning Unit: Y1
  Updated weights for Y1 (before decay applied to matrix): [0.284275 0.054675 0.038475 0.2017  ]
  All weights in matrix decayed by factor 0.5.
Weights matrix after processing Input Vector 3 and applying decay:
 [[0.1421375 0.0273375 0.0192375 0.10085  ]
 [0.0125    0.075     0.025     0.0125   ]]

Processing Input Vector 4: [1 1 1 0]
  Distances D(j): [2.65406621 2.7815625 ]
  Winning Unit: Y1
  Updated weights for Y1 (before decay applied to matrix): [0.22792375 0.12460375 0.11731375 0.090765  ]
  All weights in matrix decayed by factor 0.5.
Weights matrix after processing Input Vector 4 and applying decay:
 [[0.11396188 0.06230187 0.05865687 0.0453825 ]
 [0.00625    0.0375     0.0125     0.00625   ]]

Final Weights after all input vectors and decays:
 [[0.11396188 0.06230187 0.05865687 0.0453825 ]
 [0.00625    0.0375     0.0125     0.00625   ]]
```